Lokerin Vibecoding Workflow

Prepared for Ravenduck
Date: 2026-05-18

This document explains how Lokerin evolved through a practical AI-assisted workflow: rough idea, debugging, UI iteration, native builds, QA, product decisions, and future roadmap. It is written as something you can show a friend to explain what vibecoding looked like in this project.

What Vibecoding Meant Here

The process was not just typing one prompt and getting an app. It was closer to pair-building with an AI coding partner: you gave taste, product direction, real phone feedback, screenshots, errors, and priorities; the assistant inspected the codebase, made scoped changes, verified builds, and explained tradeoffs.
The strongest part of the workflow was fast iteration. You could say something like ’this gap feels too big’ or ’this text wraps in Indonesian’ and the implementation could immediately move from feeling to code.

Starting Point

The project began as an Expo React Native job application tracker with build/runtime problems.

- Expo SDK mismatch from 52 to 54.
- Missing dependencies.
- Invalid TypeScript syntax.
- Missing React hook imports.
- Metro bundling/runtime failures.
- Expo Router structure needed validation.

The first goal was stability: make the app compile, bundle, and run before product polish.

Phase 1: Stabilize The App

The assistant inspected package.json, route files, models, repositories, and current errors.
Fixes focused on compatibility with Expo SDK 54 and removing Metro blockers.

- Aligned dependencies using Expo-compatible packages.
- Fixed invalid TypeScript structures.
- Added missing imports such as React hooks.
- Checked Expo Router layout and route naming.
- Repeated typecheck, lint, and Android export checks.

This phase made the project safe to iterate on.

Phase 2: Define Product Identity

The app moved from a generic tracker into Lokerin: a calm, compact, bright, local-first job search companion.
Brand direction chosen by the user:

- Not dark-only.
- Bright, positive lime/coral style.
- Minimalist but compact.
- Futuristic but still practical.
- English and Indonesian support.
- Friendly tone: 'A friend for your job search journey.'

This was where product taste started guiding engineering decisions.

Phase 3: UI And Localization Iteration

The UI was tightened over many small passes based on live phone testing.

- Reduced top spacing across screens.
- Fixed search text wrapping in Indonesian.
- Aligned cards, chips, and containers.
- Made bottom tabs sit higher from the phone edge.
- Reworked application status chips into a compact two-row grid.
- Replaced stiff Indonesian copy with more natural phrasing.
- Fixed labels like Role/Title, Follow-up Date, Salary Range, Job Posting Link.
- Made recently updated items clickable.
- Themed save/archive dialogs instead of default system-looking popups.

This phase shows a key vibecoding pattern: the user describes what feels wrong, and the assistant translates that into layout, copy, and component changes.

Phase 4: Data And Local-First Features

The app stayed fully local-first, avoiding operational cost.
Implemented features:

- SQLite-backed application tracking.
- Interview logs with editable date/time.
- Follow-up dates using native date/time picker.
- Archive and restore applications.
- JSON import/restore.
- CSV export.
- PDF export.
- Backup reminders.
- Follow-up notifications.
- Calendar handoff for follow-ups and interviews.

The app does not need a backend server for these features. Data remains on the device unless the user exports or shares it.

Phase 5: Native Build Reality

A major lesson: not all Expo changes behave the same.

- JS-only changes can usually be tested by reloading Metro/dev-client.
- Native modules such as expo-notifications, react-native-pager-view, expo-dev-client, and expo-calendar require a rebuilt APK.
- App icon, adaptive icon, and splash changes also require rebuilding and reinstalling.

Real example: expo-calendar caused 'Cannot find native module ExpoCalendar' until a new development APK was built and installed. The real fix was not hiding the error; it was running an EAS development build so the native module existed in the app binary.

Phase 6: Navigation And Motion

The app moved from tab-only navigation to swipeable page navigation.

- First version used gesture detection and had clunky snap/flash behavior.
- Then it moved to react-native-pager-view for actual adjacent-page swiping.
- The user tested on device and reported overlap, flashing, and no-page-change issues.
- The implementation was adjusted until swiping felt polished.

This phase is a good example of using device feedback. Some UI behavior cannot be judged properly from code alone.

Phase 7: Product Polish

After the core app worked, the workflow shifted into final-product polish.

- Real Android package namespace: dev.ravenduck.lokerin.
- Placeholder asset pipeline for icon, adaptive icon, splash, and favicon.
- Store listing notes and screenshot checklist.
- Accessibility polish: screen reader labels, larger text safeguards, reduced motion support.
- Better empty states.
- Better analytics on Overview.
- Manual sorting: newest, follow-up soon, company A-Z.
- Search matching company, role, notes, location, job link, salary, and status labels.
- Duplicate application action.
- Onboarding reset.
- Privacy policy link and About section.
- Application templates with friendly EN/ID tone.

Phase 8: QA Loop

The QA loop mixed automated checks with real phone testing.
Automated checks used repeatedly:

- npm run typecheck
- npm run lint
- npx expo export --platform android
- expo-doctor when useful

Manual phone QA covered:
- Android dev build install.
- Light/dark mode.
- English/Indonesian language switching.
- Date/time picker.
- Notifications.
- Calendar handoff.
- JSON import/restore.
- CSV/PDF export.
- Swipe navigation.
- App icon/splash after rebuild.

Important Decisions

Several features were intentionally skipped to avoid bloat:
- Custom statuses.
- Kanban board.
- Contacts/source tracking.
- Local attachments.

This was a product maturity step. A complete app is not the app with every feature; it is the app with the right features working cleanly.

Current State

Lokerin is now a polished local-first job search tracker.
Core strengths:
- Calm compact UI.
- English and Indonesian.
- Offline local data.
- No account required.
- Follow-up and backup reminders.
- Calendar handoff.
- Import/restore backup flow.
- Multiple export formats.
- Overview analytics.
- Templates and duplicate action.
- Store prep docs and asset guidance.

Remaining launch items are mostly outside code: final privacy policy, final screenshots, final assets, QA pass, production build, Play Store setup.

Future Roadmap

Near future, zero operational cost:

- More PDF theme polish.
- More templates.
- More Indonesian copy refinement.
- More screenshot/store copy polish.
- Additional QA across Android screen sizes.
- Better user guide and privacy copy.

Future with operational cost or bigger complexity:

- Cloud sync.
- Login/accounts.
- AI suggestions.
- Email parsing.
- Job board integrations.
- Remote analytics.
- Subscription/pro features.

How To Explain The Workflow To A Friend

A simple explanation:

1. I started with a rough Expo app and real errors.
2. I used AI like a coding partner, not a magic button.
3. I gave it bugs, screenshots, UI feelings, and product direction.
4. It inspected the code, changed files, and ran checks.
5. I tested on my phone and reported what felt wrong.
6. We repeated until it felt like a real app.
7. Native features still needed real builds, installs, and QA.
8. The final product came from taste plus iteration, not one giant prompt.

Main Lesson

Vibecoding worked best when the user stayed involved as product owner, tester, and taste-maker.
The assistant was strongest at fast implementation, debugging, consistency checks, and explaining tradeoffs.
The user was strongest at deciding what felt good, what felt unnecessary, and what the app should become.

That combination is the real workflow: human judgment steering AI implementation.